30. Stock Span Problem in Java — Quick Notes

Based on the uploaded transcript.

Problem Statement

Given stock prices for **N consecutive days**, find the **span** for each day.

What is Stock Span?

Span of a day = Number of consecutive days (including current day)

whose prices are ≤ current day's price.

Example

Input

Price: 13 15 12 14 16 8 6 4 10 30

Output

Span : 1 2 1 2 5 1 1 1 4 10

Understanding Span

For: 13 15 12 14 16

Day 1 → 13

13

Span = 1

Day 2 → 15

13 15

Both ≤ 15

Span = 2

Day 3 → 12

15 > 12

Only current day counted

Span = 1

Day 4 → 14

14 ← Current
12 ≤ 14

15 > 14 → Stop

Span = 2

Day 5 → 16
13 15 12 14 16

All ≤ 16

Span = 5

Important Observations

Span is always at least 1.

Current day is always included.

Consecutive condition matters.

13 15 12 14

For 14:

12 ≤ 14 ✔

15 > 14 ✘

Cannot include 13.

Span = 2

Corner Cases

Increasing Prices

10 20 30 40
Span: 1 2 3 4

Decreasing Prices

40 30 20 10
Span: 1 1 1 1

Naive Approach

For each element: Move left.

Count consecutive
smaller/equal elements.

Algorithm

```
for each i
  span = 1

  j = i-1

  while(j>=0 &&
       arr[j] <= arr[i]){

     span++;
     j--;
  }
```

Complexity

Time	Space
$O(n^2)$	O(1)

Efficient Solution

Key Observation

Span depends on: Previous Greater Element (PGE)

Formula

If Previous Greater exists: Span = Current Index − Previous Greater Index
Otherwise: Span = Current Index + 1

Example

Price: 60 10 20 40 35 30 50 70 65

Index: 0 1 2 3 4 5 6 7 8

For 40

Previous Greater: 60
Indices: 3 − 0 = 3
Span: 3

For 35

Previous Greater: 40
Indices: 4 − 3 = 1
Span: 1

For 70

No Previous Greater.
Therefore: 7 + 1 = 8
Span: 8

Why Stack?

We need: Closest Greater Element on Left Side.

We process: Left → Right

Need to remember:

Current element
Previous Greater
Previous Greater's
Previous Greater

Order required:

Last Processed
should be checked first.
This is: LIFO
Hence: STACK

Stack Stores
INDEXES
NOT values.

Why?
Because formula requires: Current Index − Previous Greater Index

---

Algorithm
For each element:

Remove smaller/equal elements
```
while(!stack.isEmpty() && arr[stack.peek()] <= arr[i]){
  stack.pop();
}
```

Compute span

If stack empty: span = i + 1;
Else: span = i - stack.peek();

Push current index
```
stack.push(i);
```

Java Implementation
```
import java.util.ArrayDeque;
import java.util.Deque;

public class StockSpan {
  static void printSpan(int arr[]) {

      Deque<Integer> stack =  new ArrayDeque<>();
      System.out.print(1 + " ");

      stack.push(0);
      for(int i=1;i<arr.length;i++) {
          while(!stack.isEmpty() && arr[stack.peek()] <= arr[i]) {
             stack.pop();
          }
          int span = stack.isEmpty()? i + 1: i - stack.peek();
          System.out.print(span + " ");
          stack.push(i);
      }
  }
```

```
    public static void main(String[] args) {
        int arr[] = {60,10,20,40,35,30,50,70,65};
        printSpan(arr);
    }
}
```

Dry Run
Input: 60 10 20 40 35

60
Stack: [0]
Span: 1

10
Top: 60 > 10
Span: 1 − 0 = 1
Push: [1,0]

20
Remove 10
Top: 60
Span: 2 − 0 = 2
Push: [2,0]

40
Remove: 20
Top: 60
Span: 3 − 0 = 3
Push: [3,0]

35
Top: 40 > 35
Span: 4 − 3 = 1
Push: [4,3,0]

Output: 1 1 2 3 1

Why Remove Smaller Elements?

Suppose: 60 40 35
Next element: 38

Compare: 38 with 35
35 is smaller.
It can never become Previous Greater again.
So remove it.

This keeps stack useful.

Complexity Analysis

Time Complexity
Θ(n)
Reason: Every element:
Push → once
Pop → at most once

Total operations: ≤ 2n

Auxiliary Space
Worst Case: 60 50 40 30 20
Stack: All elements remain.
Space: O(n)

Best Case: 10 20 30 40
Stack size: 1
Space: O(1)

Important Interview Points

- Classic **Monotonic Stack** problem.
- Stack stores **indexes**, not values.
- Previous Greater Element determines span.
- Remove all **smaller/equal elements**.
- Each element is pushed once and popped once.
- Efficient solution runs in **Θ(n)** time.

Quick Revision

Concept	Remember
Span	Consecutive smaller/equal prices including current day
Naive Approach	$O(n^2)$
Efficient Approach	Monotonic Stack
Stack Stores	Indexes
Previous Greater Exists	`i - stack.peek()`
No Previous Greater	`i + 1`
Push Count	Once
Pop Count	At most once
Time Complexity	Θ(n)
Auxiliary Space	O(n)